

21世纪软件工程专业规划教材

软件工程导论（第6版）

目录

CONTENTS

- 1 软件危机
- 2 软件工程
- 3 软件生命周期
- 4 软件过程

第1章 软件工程学概述

- 迄今为止，计算机系统已经经历了4个不同的发展阶段，但是，人们仍然没有彻底摆脱“软件危机”的困扰，软件已经成为限制计算机系统发展的瓶颈。
- 为了更有效地开发与维护软件，软件工作者在20世纪60年代后期开始认真研究消除软件危机的途径，从而逐渐形成了一门新兴的工程学科——**计算机软件工程学**。

课程考核要求

- 平时作业 (15%)
- 课堂测试 (20%)
- 期末报告 (25%)
- 期末考试 (40%)

软件工程、软件项目管理和计算机其他专业课程的关系



计算机其他专业课: 是盖楼的科学、材料与原理, 决定楼的根基牢不牢。

软件工程: 是建筑施工方法, 决定楼怎么规范、高质量地盖出来。

软件项目管理: 是工地总指挥, 决定楼按时、按预算、安全盖完。

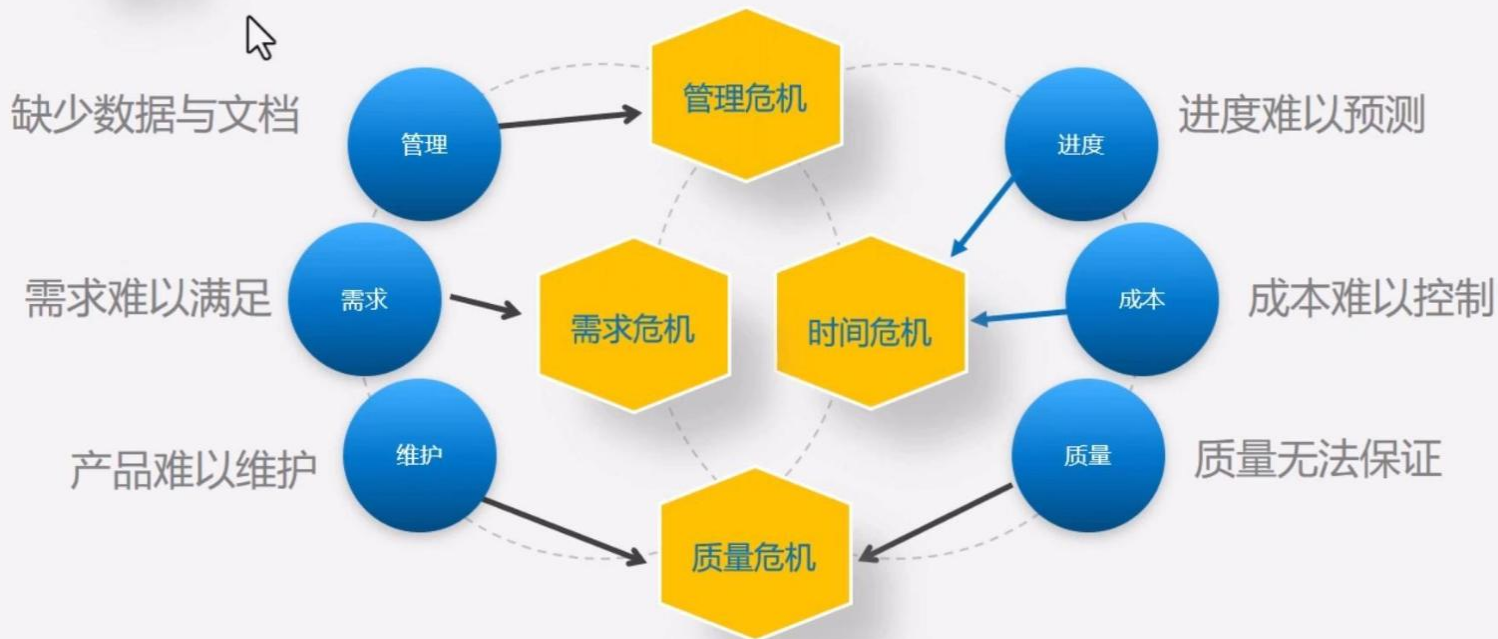
什么是软件危机？

在计算机软件的开发和维护过程中
所遇到的一系列严重问题。

<https://haokan.baidu.com/v?pd=wisenatural&vid=7966465441268709275>

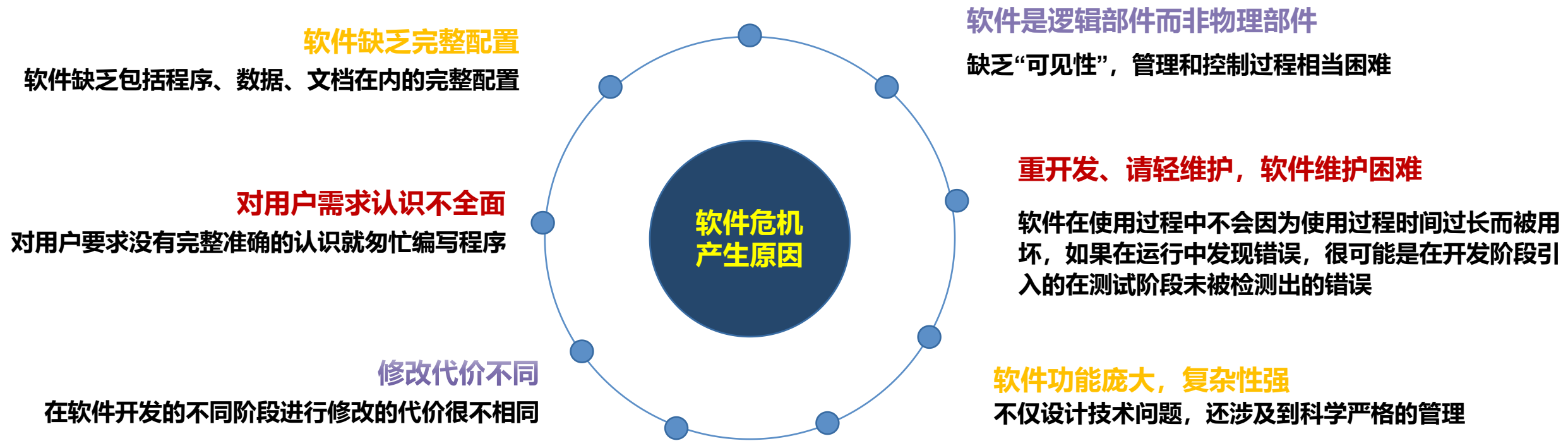


它并非指软件本身无法运行，而是指软件开发的**速度、质量、成本和管理**严重落后于硬件发展及社会对软件需求增长的矛盾状态。这个概念在20世纪60年代末至80年代被明确提出和广泛讨论，其核心是：**我们无法高效、可靠地生产出满足需求的复杂软件。**



- ① **进度难以预测**：软件开发经常不能按时完成，延期数月甚至数年。
- ② **成本难以控制**：实际花费远远超出预算。
- ③ **质量无法保证**：完成的软件漏洞百出（Bug多），不稳定，经常崩溃，难以使用。
- ④ **需求难以满足**：开发出的软件与用户想要的功能相差甚远，无法满足实际需要。
- ⑤ **产品难以维护**：软件结构混乱，修改一个错误可能引入更多错误，升级和扩展极其困难。
- ⑥ **缺少数据与文档**：缺乏文档资料会带来一系列严重且深远的问题，影响项目的短期效率和长期健康。

产生软件危机的原因



在新兴领域：

1. 技术债务：为了快速上线而牺牲代码质量，导致后期维护成本飙升。
2. 安全和隐私危机：随着软件渗透到社会各个角落，安全漏洞和隐私泄露成为新的全球性危机。
3. AI伦理与可靠性：人工智能软件的不可解释性和偏见问题，构成了新的“智能软件危机”。
4. 超大规模分布式系统的复杂性：云计算、微服务架构带来了新的运维和监控挑战。



人工智能可以极大地缓解甚至消除某些“技术性”危机，但它无法终结软件开发的“本质性”危机，甚至可能催生新的危机

AI能做到的：终结技术实现危机

-  **编码质量优化**
生成更稳定、安全的代码，自动重构，有效控制技术债务积累。
-  **知识传承与维护**
自动解释晦涩代码、生成技术文档，辅助翻译和维护遗留系统。
-  **低级错误实时纠错**
实时预判并补全代码，显著减少开发过程中的调试与试错时间。



代码所有权与法律风险

AI生成代码的知识产权归属尚不明确，可能引发潜在的法律纠纷与合规挑战。

AI做不到的：无法跨越本质鸿沟

-  **深层需求认知**
难以真正理解客户模糊需求背后复杂的商业逻辑与深层人性动机。
-  **价值权衡与担当**
虽能计算利弊，但无法承担决策带来的后果和社会责任。
-  **颠覆性创新想象**
基于已有数据的模式匹配，难以实现从0到1的真正颠覆性创新。



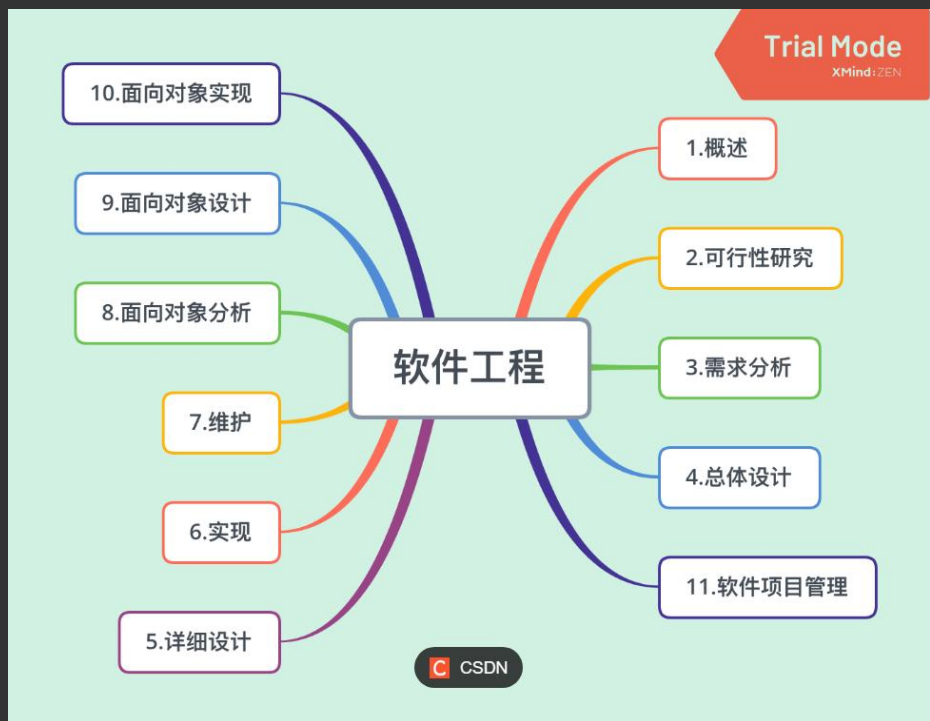
过度依赖与能力退化

长期依赖AI辅助可能导致程序员基础能力退化，系统维护的底层逻辑变得脆弱。

什么是软件工程？

为了应对软件危机，在1968年的NATO会议上，提出了“**软件工程**”的概念。其核心思想是：像**管理建筑工程、机械工程**一样，用**系统化、规范化、可量化的工程原则和方法**来指导软件开发的全过程。

软件工程是指导计算机**软件开发**和**维护**的一门**工程学科**。采用工程的概念、原理、技术和方法来开发与维护软件，把经过时间考验而证明正确的**管理技术**和当前能够得到的最好**技术方法**结合起来，以经济地开发出**高质量的软件**并有效地维护它。



软件工程的本质特性

软件工程关注于大型程序的构造

软件工程的中心课题是控制复杂性

软件经常变化

开发软件的效率非常重要

和谐地合作是开发软件的关键

必须有效地支持它的用户

为不同文化背景的人创造产品

软件工程的基本原理

1、用分阶段的生命周期计划严格管理

2、坚持进行阶段评审

3、实行严格的产品控制

4、采用现代程序设计技术

5、结果应能清楚地审查

6、开发小组的人员应该少而精

7、承认不断改进软件工程实践的必要性

软件工程方法学



方法：完成软件开发的各项任务的技术方法，回答“**怎样做**”的问题；

工具：为运用方法而提供的自动的或半自动的软件工程支撑环境；

过程：为了获得高质量的软件所需要完成的一系列**任务**的框架，它规定了完成各项任务的工作步骤。

传统方法学

也称**结构化范型**，采用**结构化技术**（结构化分析、结构化设计和结构化实现）完成软件开发的各项任务，并使用适当的软件工具或软件工程环境来支持结构化技术的运用。

特点：

- 把软件生命周期的全过程依次划分为**若干个阶段**，并顺序地完成每个阶段的任务。
- 每个阶段的开始和结束都有严格标准，前一阶段的结束标准就是后一阶段的开始标准。每一个阶段结束之前都必须进行正式严格的**技术审查**和**管理复审**。
- 每个阶段都应交出“最新式的”（即和所开发的软件完全一致的）高质量的**文档资料**，从而保证在软件开发工程结束时有一个完整准确的软件配置交付使用。
- 采用**生命周期方法学**大大提高软件开发的成功率，软件开发的生产率也能明显提高。

面向对象方法学

与传统方法**相反**，**面向对象方法**把数据和行为看成是同等重要的，它是一种以**数据为主线**，把数据和对数据的操作紧密地结合起来的方法。

4个要点：

- 把**对象(object)**作为融合了数据及在数据上的操作行为的统一的软件构件
- 把所有对象都划分成**类(class)**。
- 按照父类与子类的关系，把若干个相关类组成一个**层次**结构的系统。
- 对象彼此间仅能通过发送**消息**互相联系

优点：

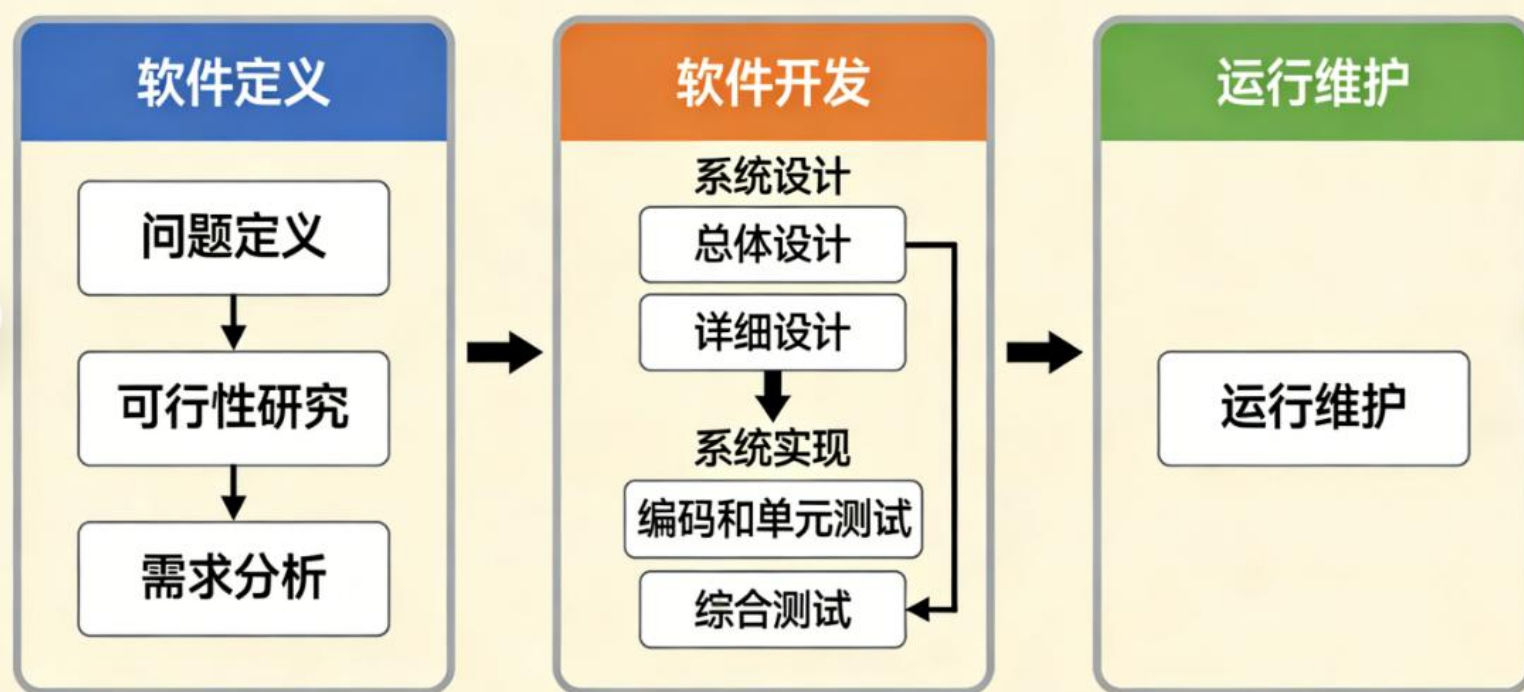
降低了软件产品的复杂性，提高了软件的可理解性，简化了软件的开发和维护工作。面向对象方法特有的**继承性**和**多态性**，进一步提高了面向对象软件的**可重用性**。

软件生命周期

软件生命周期又称为软件生存周期或系统开发生命周期，是软件的产生直到报废的生命周期。



这种按**时间分程**的思想方法是软件工程中的一种思想原则，即按部就班、逐步推进，每个阶段都要有定义、工作、审查、形成文档以供交流或备查，以提高软件的质量。但随着新的面向对象的设计方法和技术的成熟，软件生命周期设计方法的指导意义正在逐步减少。



- ① **问题定义**：要解决的问题是什么？
- ② **可行性研究**：问题是否值得去解决，是否有可行的解决办法？
- ③ **需求分析**：未解决上述问题，目标系统必须做什么？

- ① **总体设计（概要设计）**：概括来，说应该怎样实现目标系统？
- ② **详细设计**：具体来，说应该怎样实现目标系统？
- ③ **编码和单元测试**：写出正确且容易理解的程序模块
- ④ **综合测试**：预测软件的可靠性，包括集成测试和验收测试。

通过各种必要的维护活动使系统持久地满足用户的需求

软件过程

软件过程是为了获得高质量软件所需要完成的一系列任务的框架，它规定了完成各项任务的工作步骤。



软件过程定义了谁（角色）、在什么时候（阶段）、用什么方法（活动）、做什么工作（任务）、产出什么成果（制品），并如何保证质量。

“我们如何从零开始，一步步地做出一个高质量的软件？”

软件过程是软件工程的“方法论”和“操作手册”。它是在软件开发领域，将“最佳实践”制度化、系统化，以实现从混乱的“艺术创作”走向有序的“工程学科”的关键桥梁。



瀑布模型



快速原型模型



增量模型



增量模型



螺旋模型



喷泉模型



Rational统一过程



敏捷过程与基线编程



微软过程

瀑布模型一直是唯一被广泛采用的生命周期模型，现在它仍然是软件工程中应用得最广泛的过程模型。

阶段间具有顺序性和依赖性：

①必须等前一阶段的工作完成之后，才能开始后一阶段的工作； ②前一阶段的输出**文档**就是后一阶段的输入文档，因此，只有前一阶段的输出文档正确，后一阶段的工作才能获得正确的结果。

推迟实现的观点：

在编码之前设置了系统分析与系统设计的各个阶段，分析与设计阶段的基本任务规定，在这两个阶段主要考虑目标系统的**逻辑模型**，不涉及软件的**物理实现**。**清楚地区分逻辑设计和物理设计，尽可能推迟物理实现是该模型的重要指导思想。**

质量保证的观点：

软件工程的基本目标是优质、高产。为了保证所开发的软件的质量，在瀑布模型的每个阶段都应坚持：

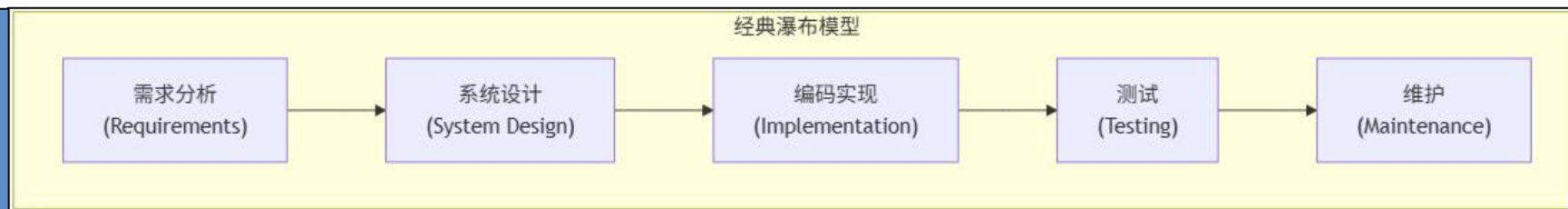
每个阶段都**必须完成规定的文档**，没有交出合格的文档就是没有完成该阶段的任务。

每个阶段结束前都要**对所完成的文档进行评审**，以便尽早发现问题，改正错误。



带有“反馈环”的瀑布模型

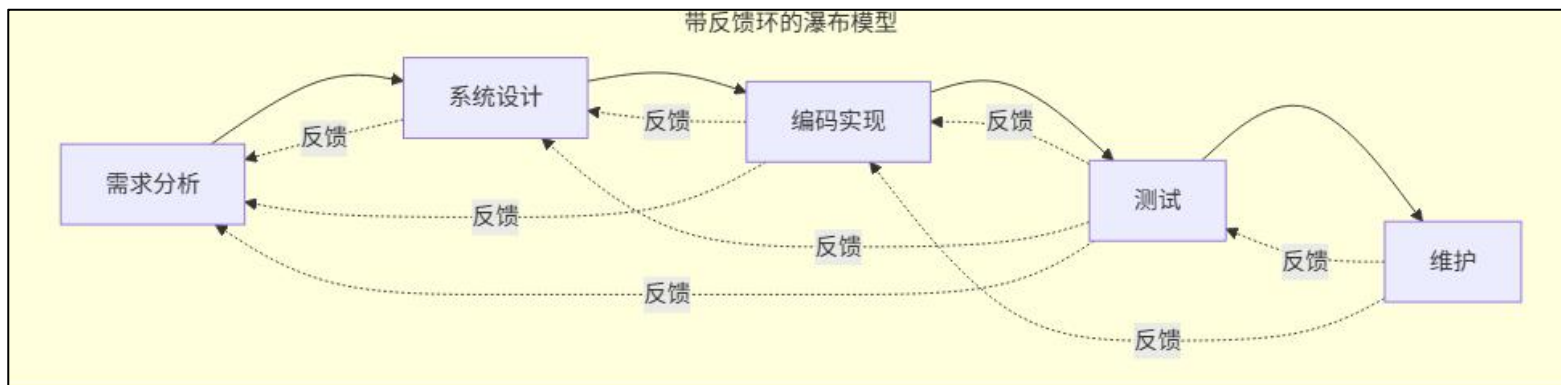
经典的瀑布模型是**线性、顺序、无回溯**的：假设每个阶段都是完美的，一旦进入下一阶段，就不能返回修改前一阶段的成果。



需求变更灾难：如果在测试阶段发现需求理解有误，必须“推倒重来”，成本极高。

缺陷堆积：前期错误到后期才发现，修复代价呈指数级增长。

僵化不灵活：无法适应变化（牵一发而动全身）。



“**反馈环**”就是指：允许从后面阶段返回到前面任何一个阶段，进行**修正和迭代**。在每个阶段结束后，或者在后继阶段发现问题时，可以沿着箭头反向回溯，回到出问题的那个阶段，进行修改。修改完成后，再重新顺序执行后续阶段。

瀑布模型有许多优点：

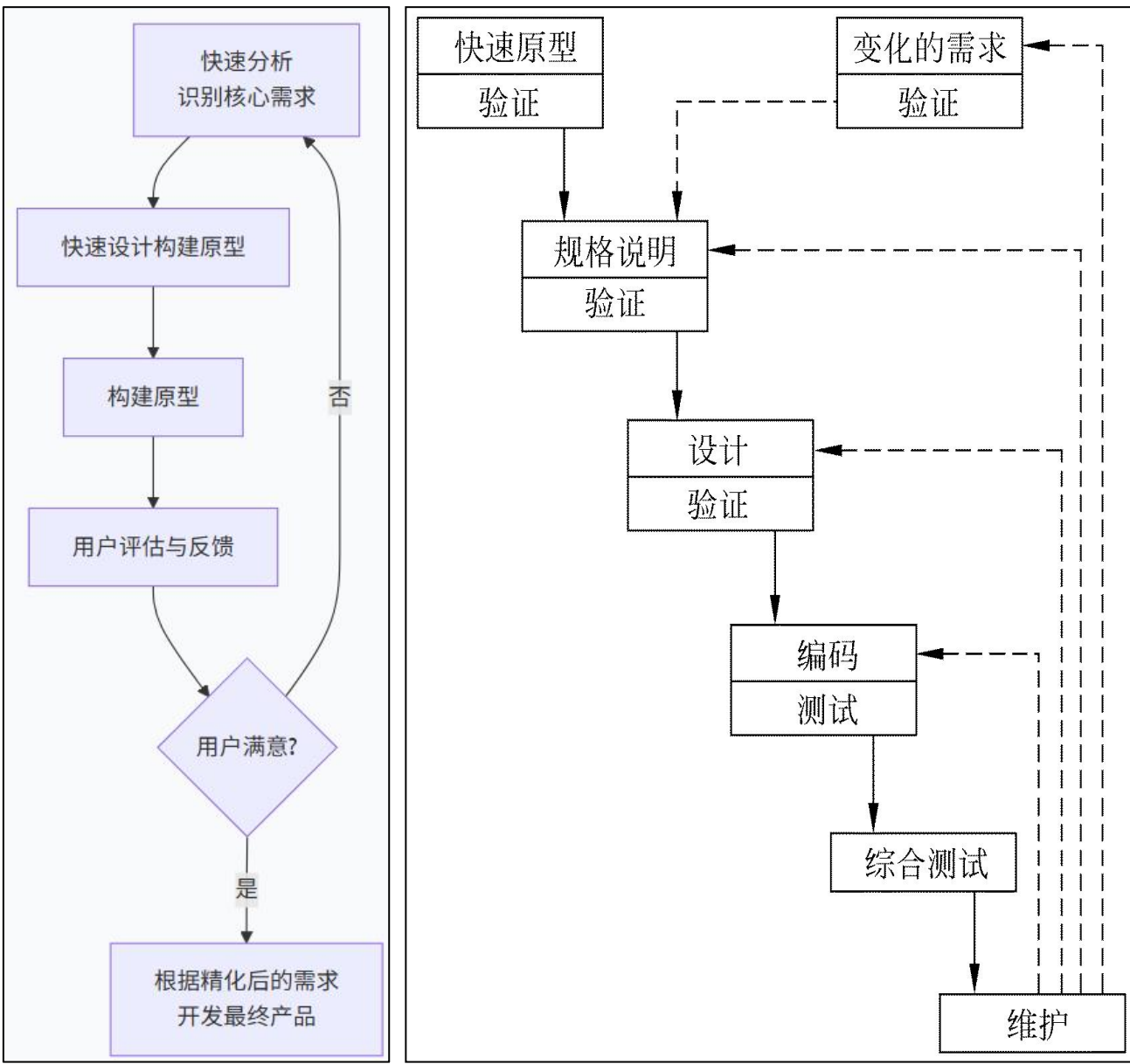
1. 可强迫开发人员采用规范的方法（例如，结构化技术）；
2. 严格地规定了每个阶段必须提交的文档，基本上是一种**文档驱动**的模型；
3. 要求每个阶段交出的所有产品都必须经过质量保证小组的仔细验证。

概念：快速原型模型是应对传统瀑布模型（特别是**需求不明确场景**）缺陷的一种重要软件开发模型。它的核心思想是：**快速构建一个简化、可运行的“原型”，让用户尽早看到并体验，从而快速澄清和精化需求。**它所能完成的功能往往是最终产品能完成的功能的一个**子集**。

特点：快速原型模型是**不带反馈环**的，这正是这种过程模型的主要优点：软件产品的开发基本上是**线性顺序**进行的。

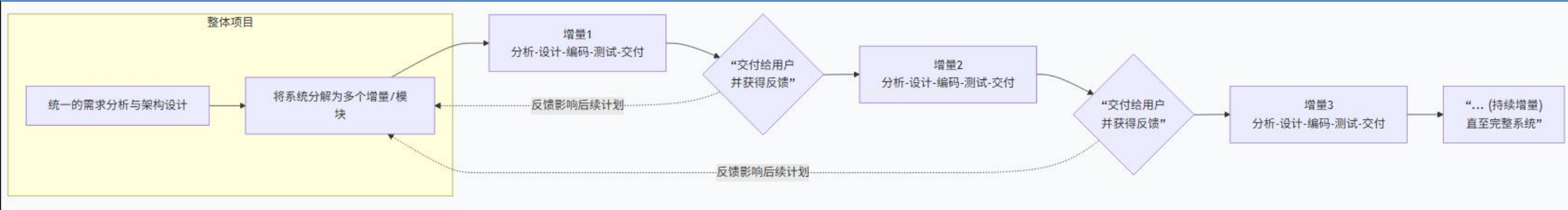
能基本上做到线性顺序开发的主要原因如下：

- （1）原型系统已经通过与用户交互而得到验证，据此产生的规格说明文档正确地描述了用户需求，因此，在开发过程的后续阶段不会因为发现了规格说明文档的错误而进行较大的返工。
- （2）开发人员通过建立原型系统已经学到了许多东西，因此设计和编码阶段发生错误的可能性也比较小，这自然减少了在后续阶段需要改正前面阶段所犯错误的可能性。



“构建原型 → 用户评估 → 快速修改” 极大降低了风险

增量模型

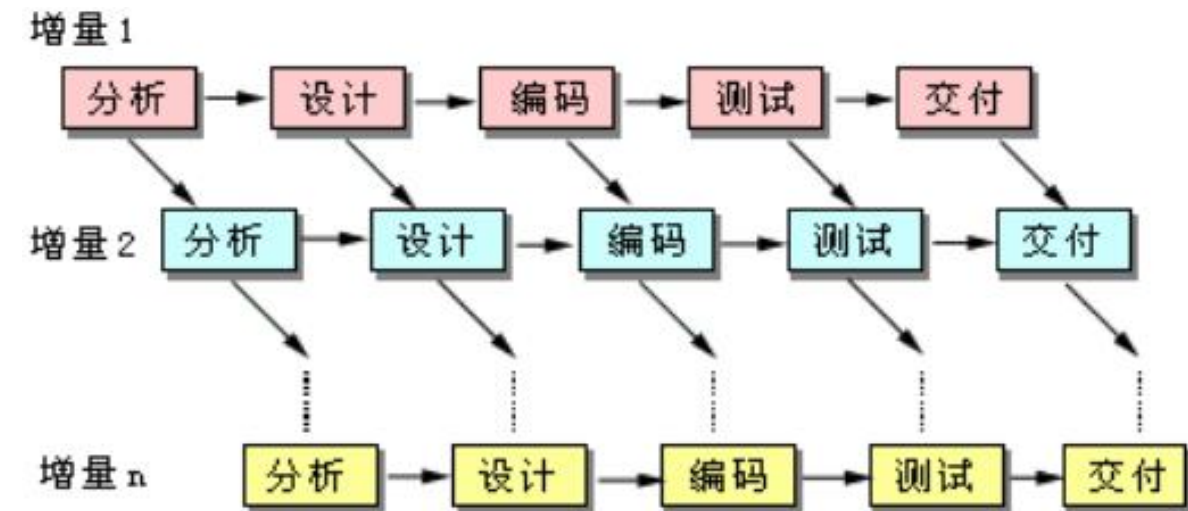


概念：增量模型也称为**渐增模型**。它的**核心思想不是一次性交付整个系统，而是分批次、渐进式地交付**。使用增量模型开发软件时，把软件产品作为一系列的增量构件来设计、编码、集成和测试。使用增量模型时，第一个增量构件往往实现软件的基本需求，提供**最核心**的功能。

优点：能在较短时间内向用户提交可完成部分工作的产品；逐步增加产品功能可以使用户有较充裕的时间学习和适应新产品，从而减少一个全新的软件可能给客户组织带来的冲击。

难点：在把每个新的增量构件集成到现有软件体系结构中时，必须不破坏原来已经开发出的产品。必须把软件的体系结构设计得便于按这种方式进行扩充，向现有产品中加入新构件的过程必须简单、方便，也就是说，软件体系结构必须是开放的。

必须在开始实现各个构建之前就全部完成需求分析和架构设计



风险更大的增量模型

前期没有一个全局的、稳健的系统架构作为基础

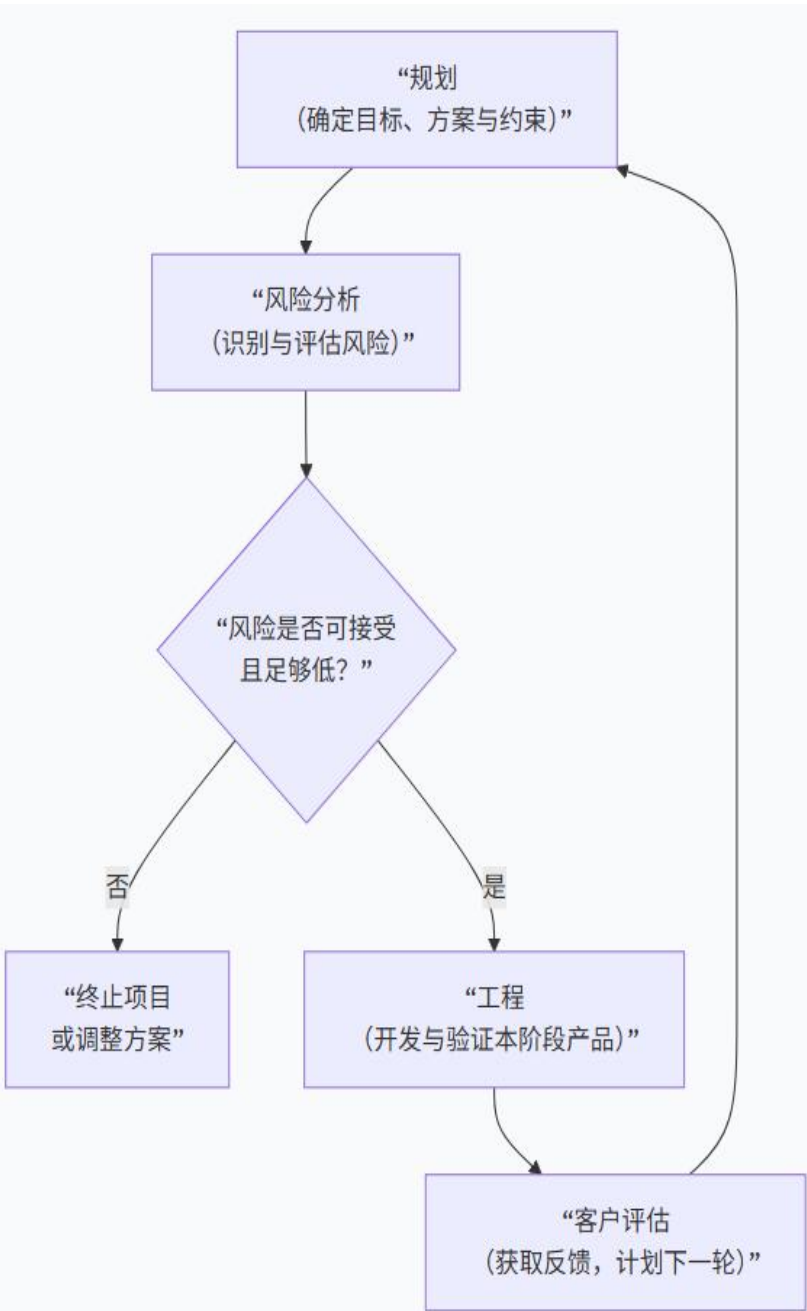
螺旋模型

概念：螺旋模型是一种**风险驱动**的软件过程模型，其基本思想是，**使用原型及其他方法来尽量降低风险**。理解这种模型的一个简便方法，是把它看作在**每个阶段之前都增加了风险分析过程**的快速原型模型。

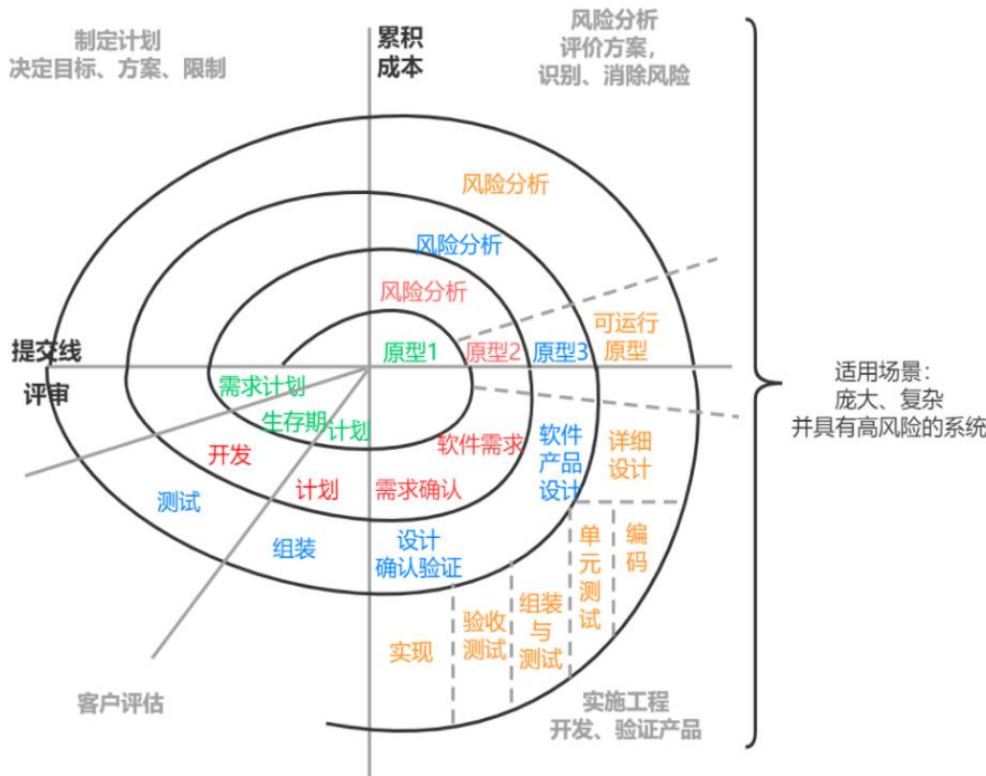
优点：

- ① 卓越的风险管理
- ② 高度的灵活性和适应性
- ③ 客户参与度高
- ④ 设计上的严谨性
- ⑤ 可早期终止

它结合了原型模型的迭代特性与瀑布模型的系统性和可控性，并将风险分析作为每个循环的核心环节。

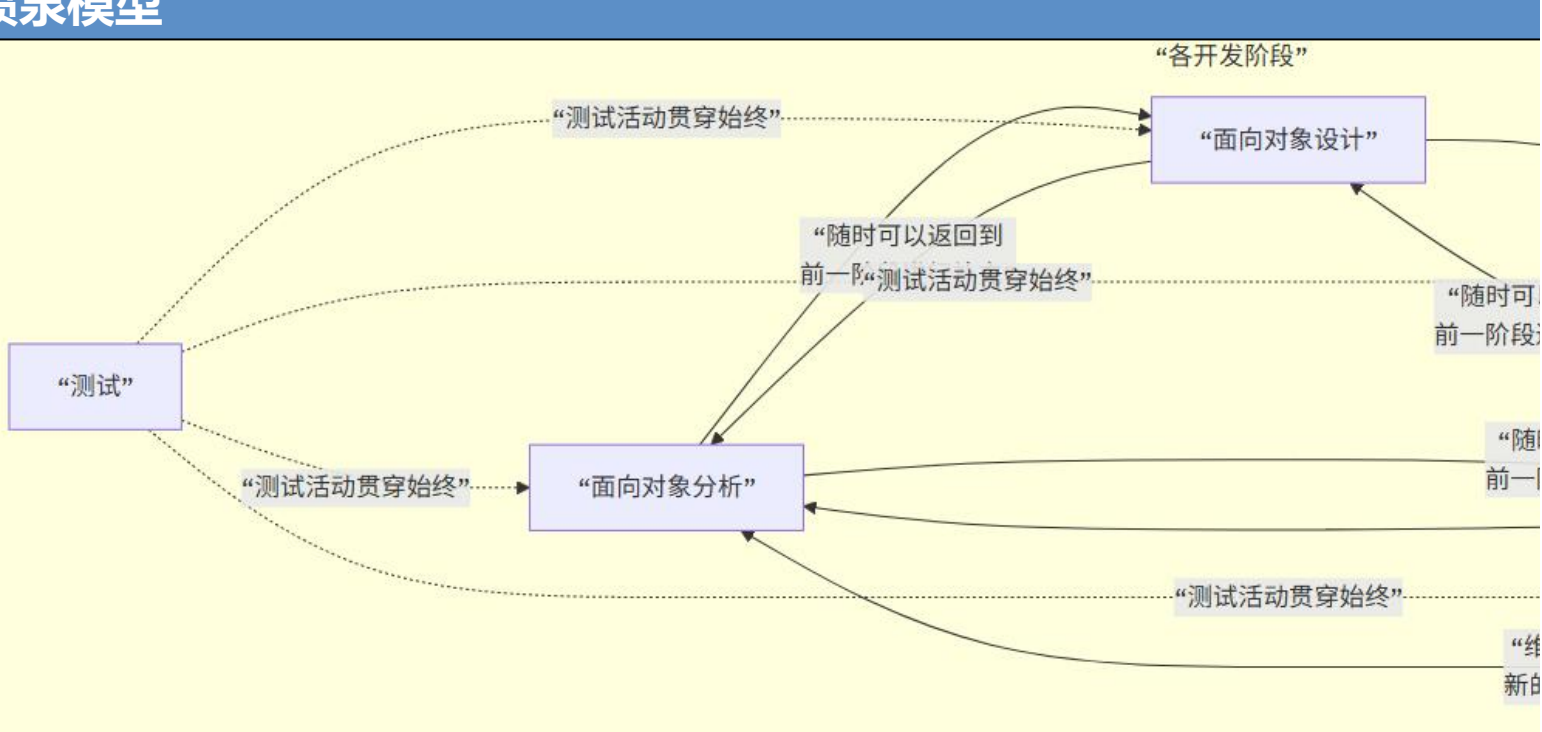


螺旋模型Spiral Model



它不像瀑布模型那样一次性完成所有事情，也不像单纯的原型法那样缺乏计划。它通过“计划 → 风险分析 → 开发 → 评估”的循环，在成本累积到不可挽回之前，通过原型和用户反馈，逐步消除风险，最终稳妥地交付高质量的产品。

喷泉模型



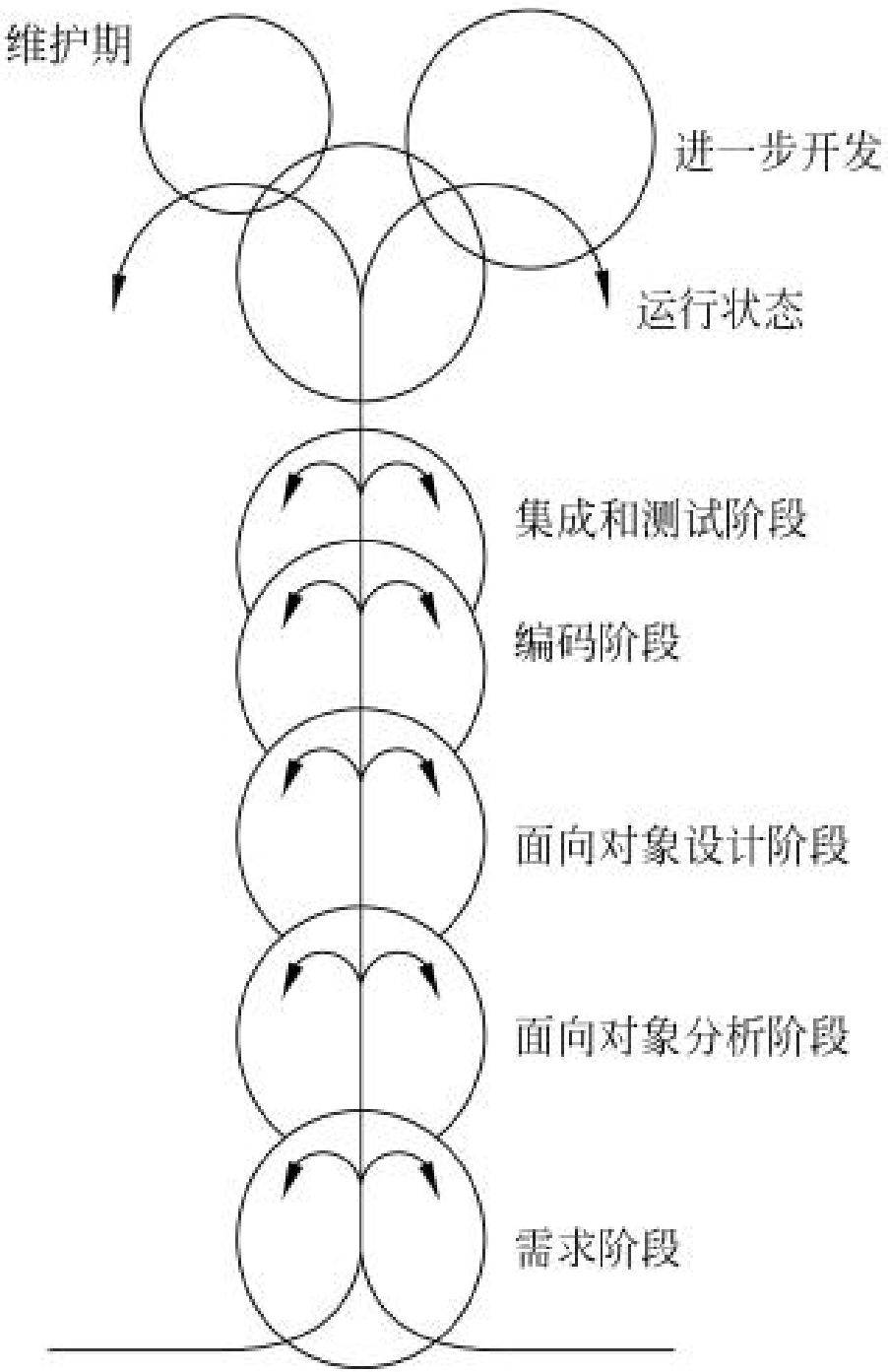
概念：喷泉模型是面向对象软件开发方法中提出的一种过程模型，它深刻地反映了面向对象开发**迭代、无间隙、无缝集成**的内在特性。用面向对象方法学开发软件时，工作重点应该放在生命周期中的分析阶段。**分析、设计和编码环节不存在明显界限。**

特点：

- ① 面向对象特性驱动的自然映射：这是喷泉模型的理论基础。
- ② 迭代性与无间隙性：这是最显著的特点。
- ③ 测试贯穿始终：测试不是一个独立的阶段，而是与所有开发活动并行进行

优点：

- ① 高效支持面向对象过程与方法的开发。
- ② 提高开发效率与质量。
- ③ 改善软件质量：持续的测试保证。
- ④ 良好的灵活性：可以方便地回溯到任何阶段，造成颠覆性影响。



概念: **Rational统一过程** (Rational Unified Process, RUP) 是由 Rational 软件公司推出的一种**完整而且完美**的软件过程。RUP总结了经过多年商业化验证的6条最有效的软件开发经验，这些经验被称为**“最佳实践”**。



迭代式开发 允许每次迭代有需求变化，采用可验证的方法减少风险

管理需求 需求是连续过程，采用用例分析捕获需求

使用基于构件的体系结构 采用现有或新构建定义体系结构

可视化建模 RUP与可视化建模语言UML紧密联系

验证软件质量 软件质量评估贯穿于整个开发过程

控制软件变更 保证每个修改都是可接受的而且能被跟踪的

RUP软件开发生命周期是一个**二维的**生命周期模型，图中纵轴代表**核心工作流**，横轴代表**时间**。

9个核心工作流

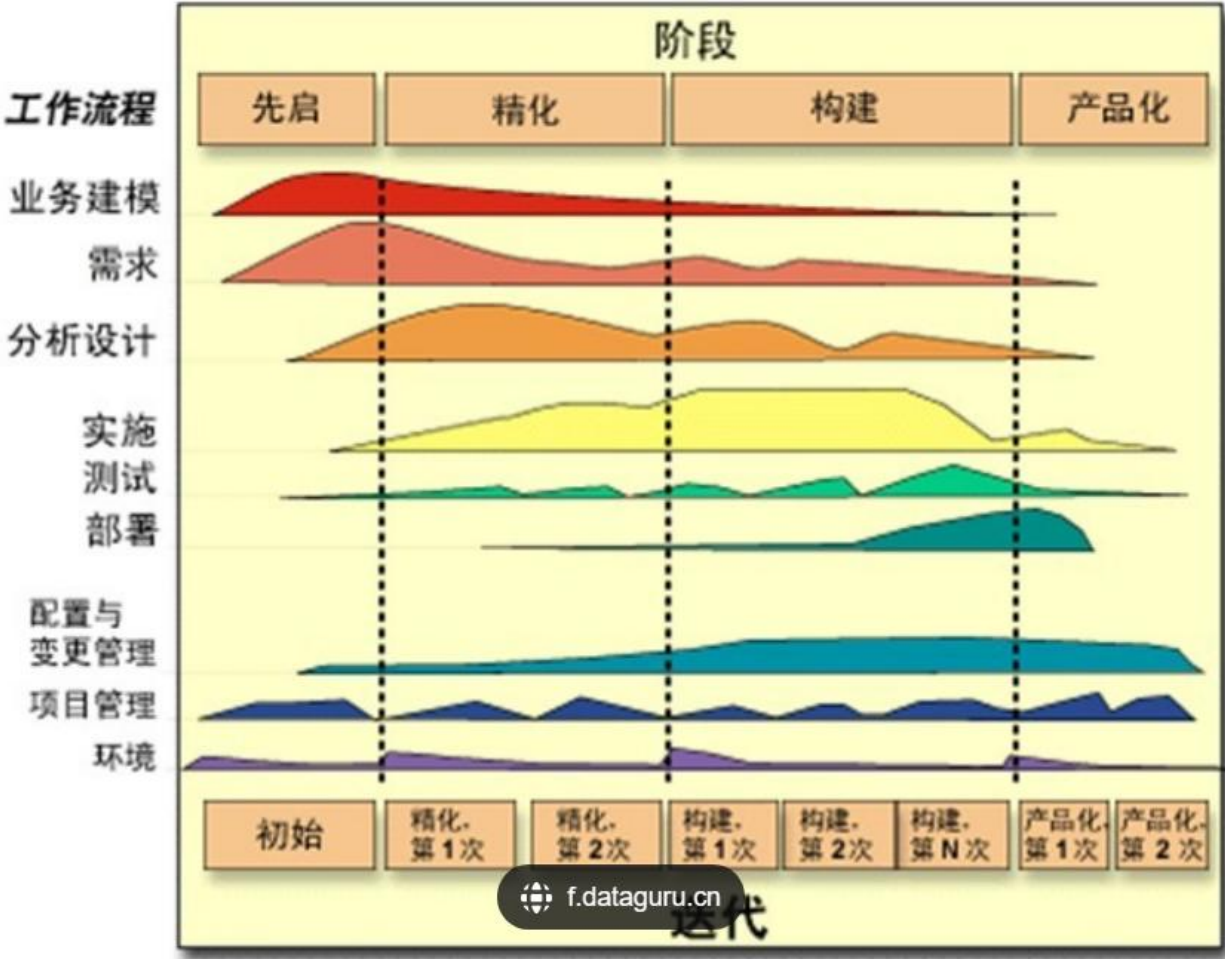
前6个是核心过程工作流，后3个是核心支持工作流

4个连续工作阶段

- ✓ 初始阶段：建立业务模型，定义产品视图，确定项目范围
- ✓ 精化阶段：设计体系结构，制订项目计划，确定资源需求
- ✓ 构建阶段：开发所有构件和应用程序并集成，测试所有功能
- ✓ 移交阶段：产品提交用户

RUP迭代式开发

强调采用迭代和渐增的方式来开发软件。开发过程由多个迭代过程组成，每次迭代只考虑系统的一部分需求。每次循环都经历一个**完整**的生命周期，每次循环结束都向用户交付产品的一个可运行的版本。每个阶段又进一步细分为一次或多次迭代过程。



每个阶段都有明确目标，采用里程碑来评估该阶段的工作

敏捷不是一种具体的方法论，而是一套价值观和原则的集合。

为了使软件开发团队具有高效工作和快速响应变化的能力，17

位著名的软件专家于2001年2月联合起草了**敏捷软件开发宣言**：

敏捷软件开发宣言由下述4个简单的价值观声明组成。

敏捷核心四大价值观

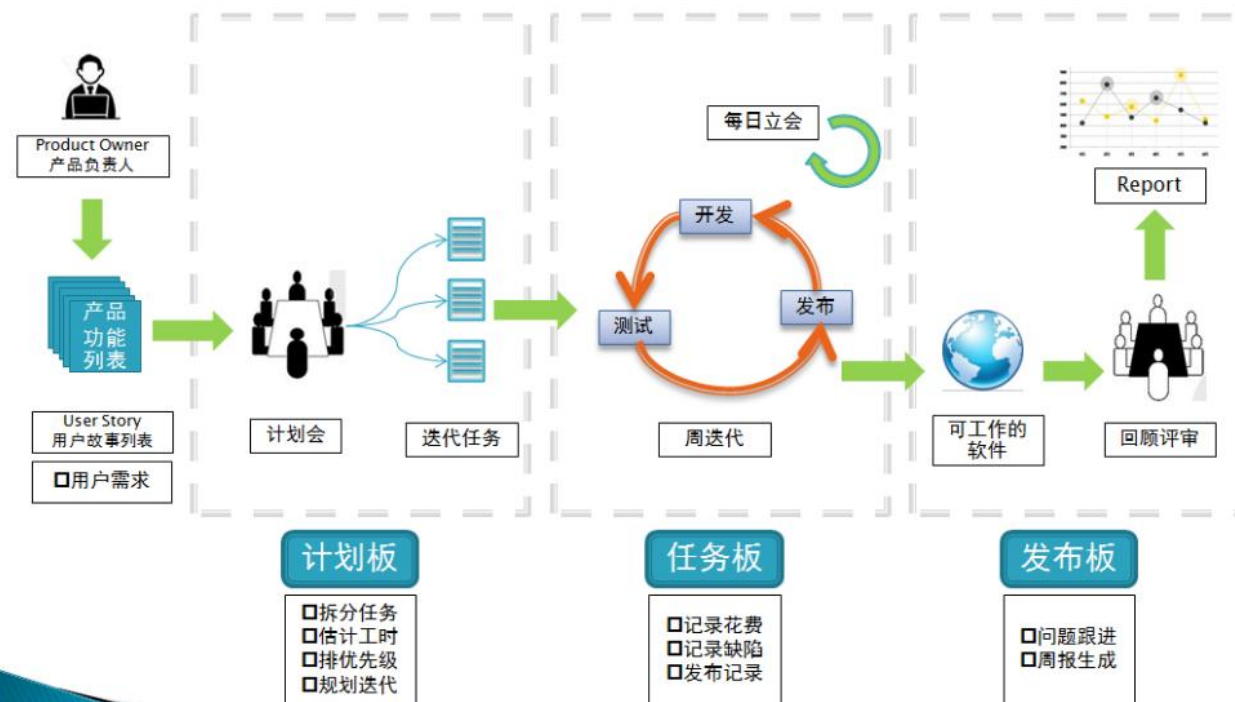
个体和交互胜过过程和工具

可以工作的软件胜过面面俱到的文档

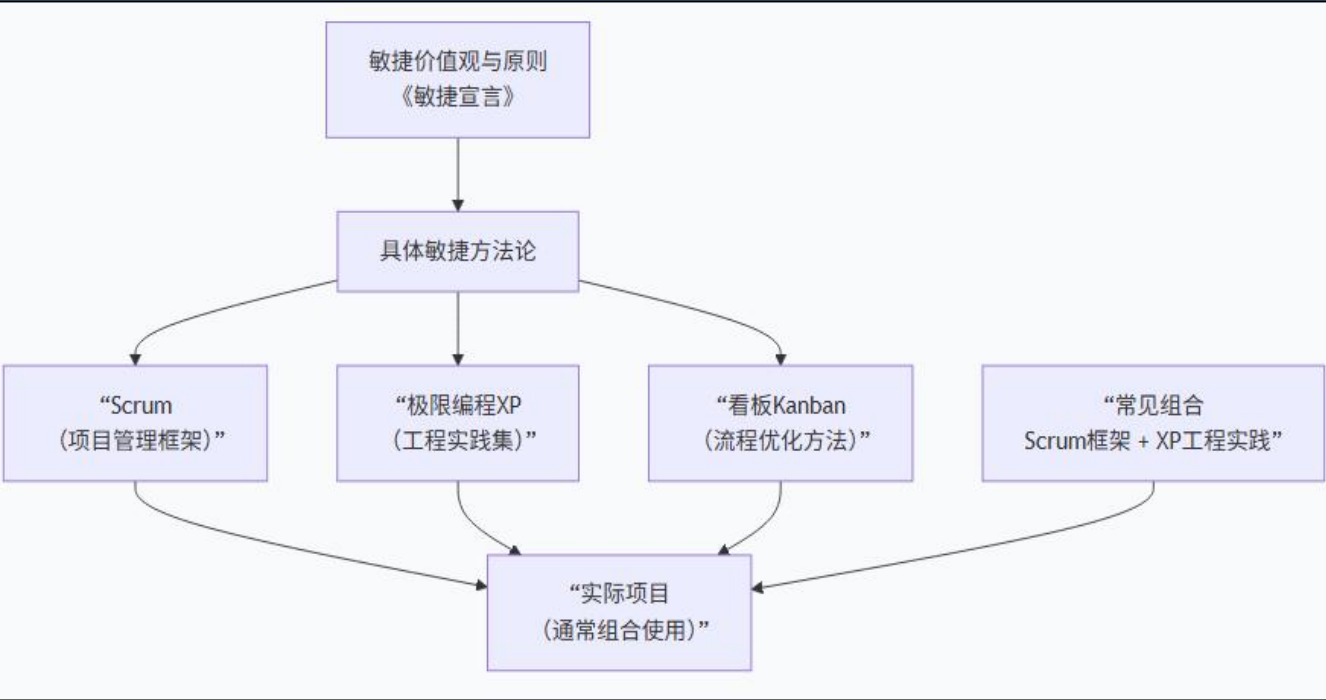
客户合作胜过合同谈判

响应变化胜过遵循计划

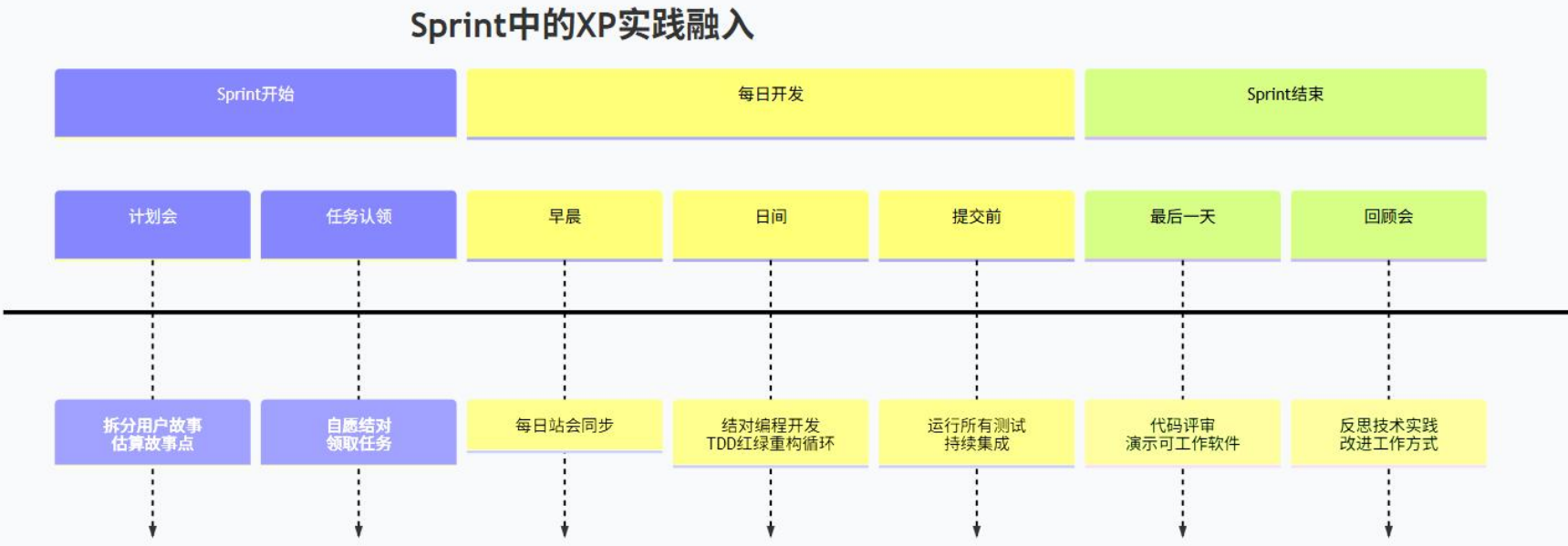
■敏捷开发流程图



- ✓ **优秀的团队成员**是最重要的因素，应首先致力于构建软件开发团队。
- ✓ 开发的主要目标是向用户提供可以工作的软件而非文档，应把主要精力放在创建软件上，**仅当迫切需要且有重大意义时才进行文档编辑工作。**
- ✓ 开发团队要与客户保持密切合作。
- ✓ 软件必须反映现实，软件过程应该有足够的能力**及时响应变化**，计划必须要有足够的可塑性和灵活性。



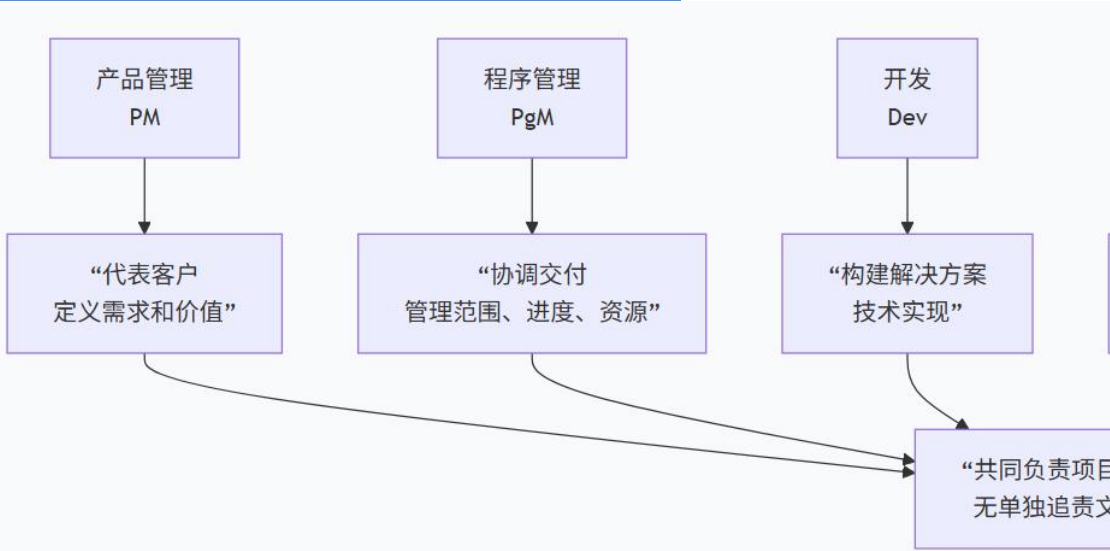
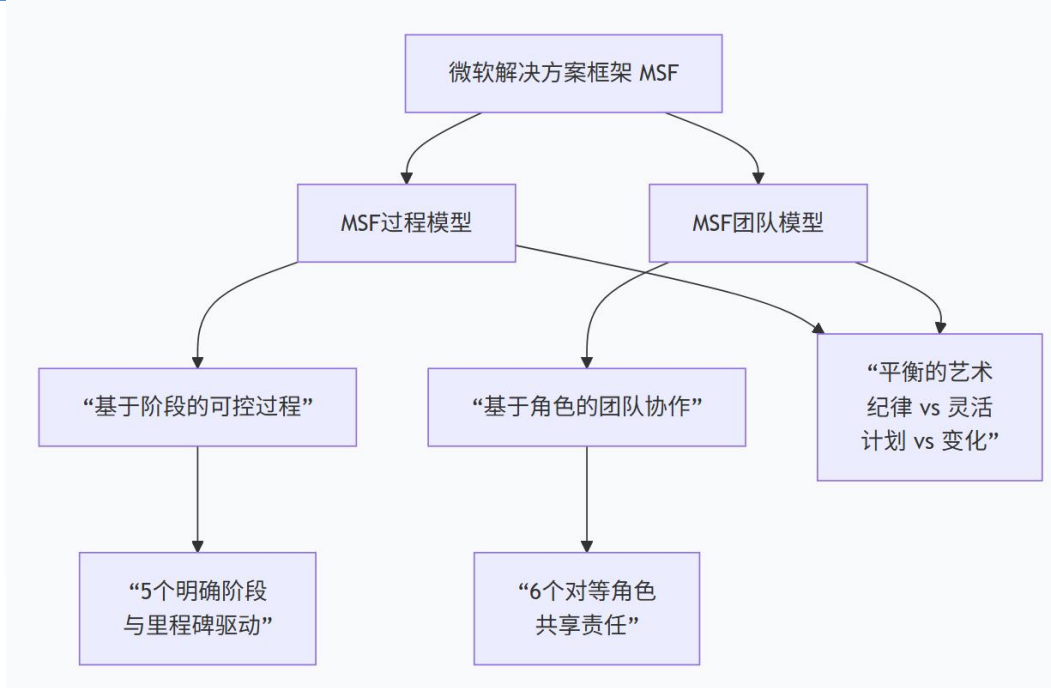
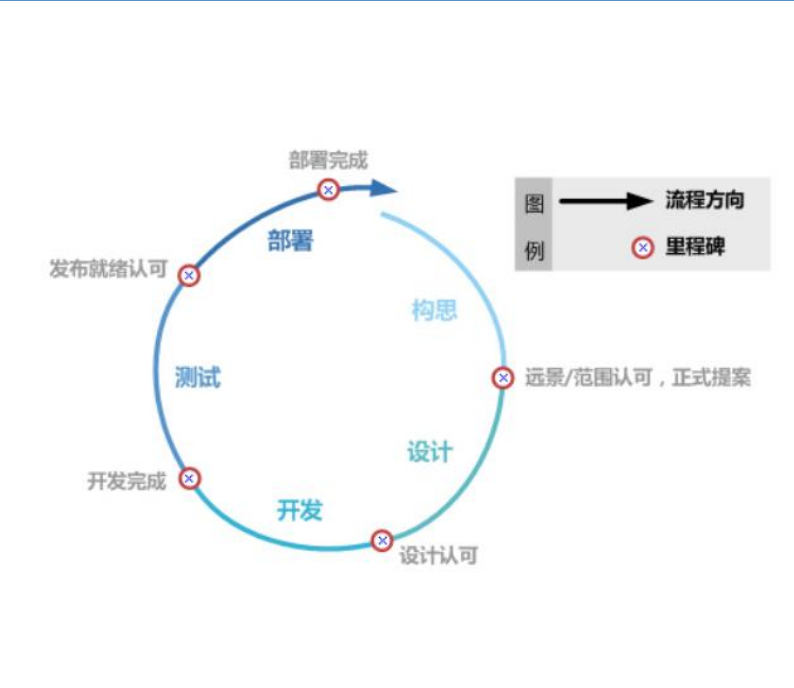
概念：极限编程（XP）是敏捷过程中最富盛名的一个，是最具体、最工程化的方法论，聚焦于软件开发实践。其名称中“极限”二字的含义是指把好的开发实践运用到极致。目前，极限编程已经成为一种典型的开发方法，广泛应用于需求模糊且经常改变の場合。



微软过程

概念：微软过程（Microsoft Solutions Framework, MSF）是微软基于自身数十年大型产品开发经验总结出的一套实用、灵活、可扩展的软件开发框架。它结合了传统工程管理和现代敏捷思想的优点。

特点：MSF过程模型是里程碑驱动的，而非严格的时间盒驱动。



特性	MSF	Scrum/敏捷	RUP
过程结构	阶段+里程碑驱动	迭代时间盒驱动	阶段+ workflow 矩阵
团队结构	6对等角色集群	3角色 (PO/SM/Team)	多种角色, 更复杂
文档要求	适度文档, 强调沟通	最少必要文档	详尽文档
变更处理	阶段内控制, 阶段间可调整	随时欢迎变化	变更控制委员会
适用规模	中小到大型项目	小到中型团队	大型复杂系统
核心优势	平衡灵活与管控	快速响应变化	系统性和完整性

 **THANKS** 